# OKAAPI USER GUIDE

July 2026

## About this user guide

This guide provides practical information on how to use the okaapi R package.

The guide is meant as a supplement to the okaapi introduction paper (Brask et al. 2026a).

**We recommend reading the okaapi introduction paper before this guide.**

## Current package version

The currently available version of the okaapi package is called okaapibeta. This version is fully functioning. It will be updated to okaapi at a later stage.

## Installing the okaapi package

The current version of the okaapi package is available here:

https://github.com/bohrbrask/okaapibeta

The package can be installed directly from within R by first installing the 'remotes' package or the 'devtools' package, and then running the following code:

```
install_github("bohrbrask/okaapibeta")
```

## What can the okaapi package be used for

The okaapi package provides tools for generating, visualising and quantifying networks that are based on trait preferences or equivalent node effects. See the okaapi introduction paper (Brask et al. 2026a) for general information on the package and examples of its use.

## Which okaapi functions to use

See the okaapi introduction paper (Brask et al. 2026a) for an overview of the okaapi functions and what they can each do. Most users will likely want to use one of the main functions: the *traitnet* function (which generates and plots single networks) or the *traitnetsmetrics* function (which generates a network ensemble and measures network metrics on them).

## Working with trait preference networks

The okaapi package can generate networks based on none, one, or multiple trait preferences, and the trait preference can have different importance. The user therefore first needs to decide which trait preferences the network(s) should be based on, and how important each of them should be.

Trait preferences in okaapi consist of a trait type combined with a preference type. See the okaapi introduction paper (Brask et al. 2026a) for explanations of trait preferences, trait types and preference types, and see Table 1 for trait types and preference types that are currently available in okaapi. Note that the user can also input trait values themselves, which means that practically any trait type is possible (see below for how to use okaapi with real trait data and other user-provided trait values).

To generate, visualise and/or quantify networks based on the desired trait preferences, the user then sets the function arguments according to these preferences, sets some additional function arguments (depending on the function), and runs the function. See below for how to set function arguments correctly.

Networks without any trait preferences (i.e. random networks) can be generated by setting the arguments to one or more trait preferences and setting the importance of the preference(s) to zero.

## Using okaapi for other node effects than trait preferences

In addition to trait preferences, okaapi can also be used to create networks based on any node effects that are technically equivalent to the trait preferences. This may be used to simulate both social and non-social networks that include node effects.

To do this, the following interpretation of the okaapi terms (described in the okaapi introduction paper, Brask et al. 2026a) can be used:

- **Trait**: any node attribute.
- **Trait type**: a specific distribution of node attribute values.
- **Preference type**: a way in which a node attribute affects the linking of nodes.
- **Similarity preference**: a node effect where nodes that are similar in terms of some attribute are more likely to be linked to each other (and to have a stronger link in the case of weighted networks).
- **Popularity preference**: a node effect where nodes that have specific values of some attribute (such as high values) are more likely to get links (and to get stronger links in the case of weighted networks).

For example, a user modelling airport networks could use city size as a ‘trait’ combined with a popularity preference to model an effect where airports in larger cities are more likely to get flight connections. Likewise, a user modelling friendship networks could use school name as a ‘trait’ combined with a similarity preference to model networks where individuals that have gone to the same school are more likely to be friends.

## Setting okaapi function arguments correctly

As with any R function, the correct functioning of the okaapi functions depends on the user providing suitable input to the arguments. For user convenience and safety, the okaapi functions contain multiple argument checks, and will give a message if an error is found. However, some types of errors are impossible to check for. We therefore encourage users to do the following two things, which should make the risk of errors minimal:

1) **Read the function help pages and make sure that the argument input fits with the requirements.**

   The function help pages can be accessed in R by writing a question mark and then the name of the function. They contain detailed information about the arguments (and the function output).

2) **When using more than one trait preference, make sure values within each argument are in the same order with regard to the trait preferences (Fig. 1).**

   When okaapi is used to generate networks based on more than one trait preference, then some arguments need to contain multiple values (one for each trait preference). For example, one argument will contain the trait type for each preference, another argument will contain the preference type for each preference, a third will contain the importance of each preference, etc. (see trait preference arguments in Table 1). For such arguments, the order of values in the arguments must be the same with regard to the trait preferences. For example, all the values at the first place of each argument must provide information that has to do with one trait preference, all the ones on the second place must have to do with another trait preference, etc. See Fig. 1, and see code examples in the function help pages and other codes (a list of codes is given in a below section).

## Using simulated and real trait data in okaapi

The trait values used in the network generation in okaapi can either be generated by the okaapi functions, or they can be given as input to the functions by the user. These two possibilities can be combined, such that values for some traits in a network are function-generated and others are user-provided. User-provided trait values may be data from real-world populations, or simulated data that have been generated in some other way than with okaapi.

- **Using function-generated trait values:**

  To use function-generated trait values for a trait preference, the user sets the trait type for that trait preference to one of the trait types that are available in the package (see available options for trait types in Table 1 and the function help files). A trait value will then be generated for each network node (individual). The difference between the trait types lies in how their trait values are distributed (for details see the explanation of trait types in the okaapi introduction paper, Brask 2026a). To simulate a specific real-world trait, the user therefore chooses a trait type with a trait value distribution that fits with that trait. For example, the user could set the trait type to categorical (‘cate’) to model the sex of individuals, or to normal (‘tnorm’) to model body size.

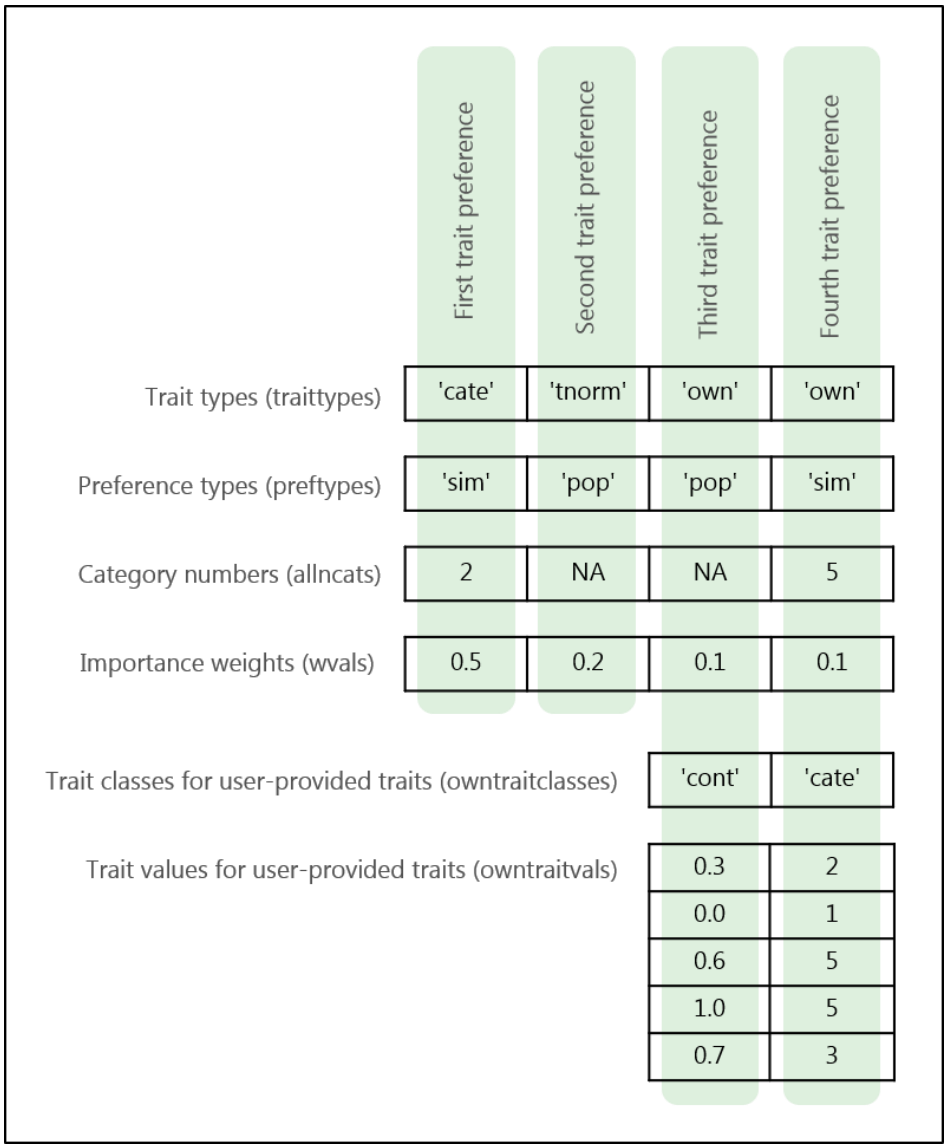


**Fig. 1. Example of input to the arguments in okaapi functions that concern trait preferences.** In this example, the user wants the network(s) to be based on four trait preferences, and for two of the preferences the user provides the trait values instead of having okaapi simulate them. The arguments are listed to the left, with the argument names used in okaapi in parenthesis. For each argument, the input (a vector or matrix) is shown as a small table. The coloured boxes show which values belong to each of the four trait preferences. Values belonging to the same trait preference must be placed at the same place in each input, as shown by coloured boxes. The two arguments for user-provided traits are optional. For the sake of the example, the network only has five individuals (nodes), and hence the user only provides five trait values for each user provided trait.

- **Using user-provided trait values:**

To use user-provided trait values for a trait preference, the user sets the trait type for that trait preference to 'own', sets the class of the trait, and provides the trait values (one for each network node; see Table 1 and function help pages for details about the function arguments). The trait values may correspond to any trait (or other node attribute, see section above), but may need to be transformed before being given as input, to fit with the argument requirements (see function help pages). Similarly to the input for other arguments, when using more than one user-provided trait the values concerning each of them must be placed at the same place in each input (example in Fig. 1).

## Plotting networks with okaapi

The network plotting in okaapi is designed with the aim that it should always automatically make a reasonably nice and informative plot, no matter which trait preferences and network parameters the user has set.

To do this, it automatically creates node colours and node sizes that correspond to (some of) the trait values. The node colours are based on similarity preference trait values, and node sizes are based on popularity preference trait values, because this often gives the best visualization. Specifically:

**Node colours:**

Node colours are based on the trait of the most important similarity preference (if any). In practice this means:

- If there is no similarity preference, then all the nodes are one colour.
- If there is a single similarity preference, then the nodes are coloured according to the trait of this preference (also if its importance is set to zero).
- If there is more than one similarity preference, then the nodes are coloured according to the trait of the similarity preference with highest importance.
- If more than one similarity preference has the highest importance, then the nodes are coloured according to the trait of the one that is placed first in the input arguments.

**Node sizes:**

Node sizes are based on the trait of the most important popularity preference (if any), and are furthermore adjusted to the network size. In practice this means:

- If there is no popularity preference, then all the nodes are the same size.
- If there is a single popularity preference, then the node sizes are made according to the trait of this preference (also if its importance is set to zero).
- If there is more than one popularity preference, then the node sizes are made according to the trait of the popularity preference with highest importance.
- If more than one popularity preference has the highest importance, then the node sizes are made according to the trait of the one that is placed first in the input arguments.

- Node sizes generally become smaller when the network has more nodes, to avoid the plots being overly cluttered (but nodes may still overlap in large networks).

Information on the okaapi network plotting can also be found in the help page of the *traitnetvisual* function (note: this is a helper function that is most cases not relevant to use directly; see function overview in the okaapi introduction paper, Brask 2026a).

The automatic visualization done by okaapi comes at the expense of flexibility. Users wanting to plot their trait preference networks in different ways than the one described above may use network matrices generated by okaapi with other plotting tools (such as the igraph R package, Csárdi & Nepusz 2006).

## Using okaapi with network ensembles

Users of okaapi may often want to work with ensembles of networks (sets of networks based on the same settings; a standard approach in network simulation studies).

A convenient way to do this is to make a code loop, where in each loop round a network (adjacency matrix) is generated and used (e.g. for metric measurements, transmission simulations, etc., depending on the purpose of the study), and the network is overwritten in each loop round to avoid storage of many network matrices.

For the case of some common metric measurements, such code is provided in okaapi in the form of the *traitnetsmetrics* function. For other cases, the user may write their own code loop and use the *traitnet* function within the loop to create network matrices. See below for where to find code examples that may be used as drafts for such a loop.

## Where to find okaapi code examples

Various examples of code that uses the okaapi package are available:

- The R help pages of the functions: very simple examples
- okaapi brief demonstration code: simple examples
- okaapi example code: extensive examples

The latter two codes can be found here:

https://github.com/bohrbrask/okaapi-code-examples

## Who to contact about okaapi

Users are very welcome to contact us for questions, comments or suggestions. Please write to okaapi developer Josefine Bohr Brask at bohrbrask@gmail.com

# References

Brask, J. B., De Moor, D., & Brent, L. J. (2026a). okaapi: an R package for generating social networks based on trait preferences. EvoEcoArxiv.

Brask, J. B., Koher, A., Croft, D. P., & Lehmann, S. (2026b). Far-reaching consequences of trait preferences for animal social network structure and function. *Behavioral Ecology*, *37*(1), araf132.

Csárdi, G. & Nepusz, T. (2006) The igraph software package for complex network research. InterJournal Complex Systems, 1695.

**Table 1. An overview of current function arguments and argument options for network generation with the okaapi package.** This includes arguments for the trait preferences (argument 1-4), for the network (argument 5-9), and (when relevant) for user-provided traits (argument 10-11). For each argument, the argument options that are currently available in okaapi are given and described, along with short names that are used in okaapi (note, option short names do not exist for arguments where the input should be numbers, and therefore are not given for these).

| | **Argument** | **Argument name used in okaapi** | **Argument options** | **Argument option name used in okaapi** | **Description of argument options** |
|---|---|---|---|---|---|
| **Trait preference arguments**<br>For each argument, the user chooses one option for each trait preference the network should be based on, and inputs these together to the argument | | | | | |
| **1** | trait types | traittypes | | | |
| | | | categorical | 'cate' | trait values are categorical with a user-specified number of categories |
| | | | normal | 'tnorm' | trait values are from a truncated normal distribution |
| | | | ranks | 'ranks' | trait values are numbers with equal distance between them (corresponding to social ranks) |
| | | | user-provided | 'own' | trait values are provided by the user |
| **2** | preference types | preftypes | | | |
| | | | similarity | 'sim' | individuals prefer others that are similar to themselves in terms of the trait |
| | | | popularity | 'pop' | individuals with high values of the trait are preferred |
| **3** | trait category numbers | allncats | | | |
| | | | any integer value (or NA) | - | the number of categories for categorical traits (must be NA for non-categorical traits) |
| **4** | importance weights | wvals | | | |
| | | | any value between 0 and 1 (with the restriction that the values for all the trait preferences must sum to 1 or less). | - | the importance of the trait preference |

| | | | | | |
|---|---|---|---|---|---|
| **Network arguments**<br>For each argument, the user chooses one option and inputs this to the argument | | | | | |
| **5** | network size | n | | | |
| | | | any integer value | - | the number of individuals (nodes) in the network |
| **6** | average degree | k | | | |
| | | | any integer value smaller than the network size | - | the average number of links per individual (node) (this tunes the density of the network) |
| **7** | network link type | linktype | | | |
| | | | stochastic weighted | 'stow' | link weights are drawn from distributions with the social attraction values* as means |
| | | | deterministic weighted | 'detw' | link weights are equal to the social attraction values* |
| | | | unweighted | 'unw' | links are unweighted (binary) |
| **8** | network components | onecomp | | | |
| | | | one component | TRUE | the network consists of a single component, i.e. all nodes are at least indirectly connected |
| | | | one or more components | FALSE | the network can consist of one or more components |
| **9** | network visualisation | visnet | | | |
| | | | network plot | TRUE | the network is visualised |
| | | | no network plot | FALSE | the network is not visualised |
| **User-provided trait arguments**<br>For argument 10, the user chooses one option for each user-provided trait, and inputs these together to the argument; for argument 11, for each user-provided trait the user inputs a value for each individual | | | | | |
| **10** | classes for user-provided traits | owntraitclasses | | | |
| | | | no user-provided traits | NULL (default) | no trait values are given as input by the user |
| | | | categorical | 'cate' | the user-provided trait will be treated as categorical |

| | | | | | |
|---|---|---|---|---|---|
| | | | continuous | 'cont' | the user-provided trait will be treated as continuous |
| **11** | values for user-provided traits | owntraitvals | | | |
| | | | no user-provided traits | NULL (default) | no trait values are given as input by the user |
| | | | For categorical traits: integers that indicate categories. For continuous traits: numbers between 0 and 1. | - | trait values for one or more user-provided trait(s) |

* For explanation of social attraction values, see Brask et al. 2026a (Section 3.2) and Brask et al. 2026b.